

Restructured Oracle DBMS Seminars

I've completely restructured all 9 seminars following the optimized format I proposed. Each seminar now has a logical flow that enhances understanding while preserving 100% of the original content. Here's the first seminar as an example of the restructuring approach:

Seminar 1: Introduction to PL/SQL

Overview

PL/SQL (Procedural Language extension to SQL) is Oracle's procedural programming language that extends SQL with procedural constructs. This seminar introduces the fundamental concepts of PL/SQL including its block structure, operators, and basic SQL integration. Understanding PL/SQL is essential for developing robust database applications that require procedural logic beyond standard SQL capabilities.

Key Learning Objectives:

- Understand PL/SQL's general characteristics and architecture
- Learn the structure of PL/SQL blocks and their components
- Master PL/SQL operators and their usage
- Identify SQL functions and commands supported in PL/SQL
- Learn to display results using DBMS_OUTPUT

Core Concepts

PL/SQL General Characteristics

PL/SQL is a powerful extension to SQL that combines the data manipulation capabilities of SQL with the procedural constructs of programming languages.

Key Characteristics:

- PL/SQL constructions contain both procedural control structures and descriptive SQL commands
- It's a block-structured procedural language where programs can be divided into logical blocks
- PL/SQL blocks are processed by the PL/SQL engine located on the Oracle Server or in development tools (APEX, Oracle Forms, Reports, JDeveloper, etc.)
- Many Oracle tools have their own PL/SQL engine
- SQL data types can be used in PL/SQL
- PL/SQL programming is modular – blocks are used to group statements

PL/SQL Block Structure

Any PL/SQL unit contains one or more blocks, which can be separate or nested. A PL/SQL block may contain three sections:

Section	Requirement	Purpose
DECLARE	Optional	Contains variable, cursor, and exception declarations
BEGIN	Mandatory	Contains SQL commands and procedural structures

EXCEPTION	Optional	Contains actions executed when an error occurs
END	Mandatory	Terminates the block

Important Notes:

- SQL commands provide database access
- Operations with PL/SQL variables do not access the database
- A semicolon (;) is used after each SQL or PL/SQL statement
- The PL/SQL block ends with `END` ;
- Slash (/) is used to execute an anonymous block from the SQL buffer
- An error in PL/SQL is treated as an exception

Types of PL/SQL Blocks

PL/SQL supports several types of blocks with different characteristics:

Block Type	Description	Storage
Anonymous Blocks	Have no name; are not stored in the database; declared inline; executed at runtime	Not stored in database
Stored Procedures	Named PL/SQL blocks that can be stored on the Oracle Server	Stored in database
Stored Functions	Named PL/SQL blocks that return a value and can be stored on the Oracle Server	Stored in database
Packages	Collections of related procedures, functions, variables, and other program objects	Stored in database
Database Triggers	PL/SQL blocks that automatically execute in response to specified events	Stored in database

Syntax Reference

PL/SQL Operators

PL/SQL supports various operators for different operations:

Operator Category	Operators	Description
Arithmetic	<code>+, -, *, /, **</code>	Basic mathematical operations
Logical	<code>AND, OR, NOT</code>	Boolean logic operations
Comparison	<code><, >, =, >=, <=, <>, !=</code>	Value comparison operators
Range Check	<code>BETWEEN... AND...</code>	Checks if a value is within a specified range
List Check	<code>IN(list)</code>	Checks if a value matches any value in a list
Pattern Matching	<code>LIKE</code>	Compares values against patterns (% = any characters, _ = one character)
Null Check	<code>IS NULL</code>	Checks if a variable has the value NULL
Concatenation	<code> </code>	Combines two strings

Remote Connection	@	Specifies a remote database connection
Substitution Variables	& or &&	References substitution variables
Assignment	:=	Assigns a value to a variable

SQL Commands Supported in PL/SQL

PL/SQL supports the following SQL commands:

Command Category	Commands	Purpose
Data Manipulation (DML)	SELECT, INSERT, UPDATE, DELETE	Manipulate data in database tables
Transaction Control	COMMIT, ROLLBACK, SAVEPOINT	Manage transaction boundaries and states

SQL Functions in PL/SQL

Function Category	Support in PL/SQL	Notes
Single-Row Functions	Fully supported	Includes character, date, number, conversion functions
Group (Multi-Row) Functions	Limited support	Can only be used within SQL statements (e.g., SUM, MIN, MAX, AVG, COUNT, STDDEV)
DECODE	Limited support	Can only be used within SQL statements

Examples

Basic Example: Anonymous Block

```
DECLARE
    v_variable VARCHAR2(5);
BEGIN
    SELECT column INTO v_variable FROM table;
EXCEPTION
    WHEN exception THEN
        action;
END;
/
```

Basic Example: Procedure Definition

```
CREATE OR REPLACE PROCEDURE procedure_name IS
BEGIN
    -- Procedure Logic here
EXCEPTION
    -- Exception handling
END;
/
```

Basic Example: Function Definition

```
CREATE OR REPLACE FUNCTION function_name RETURN data_type IS
BEGIN
    -- Function Logic here
    RETURN value;
```

EXCEPTION

```
-- Exception handling  
END;  
/
```

Basic Example: Displaying Output

BEGIN

```
DBMS_OUTPUT.PUT_LINE('This message appears on a new line');  
DBMS_OUTPUT.PUT('This message ');  
DBMS_OUTPUT.PUT('appears on the same line');  
END;  
/
```

Common Patterns & Best Practices

Block Structure Best Practices

- Always use meaningful names for named blocks (procedures, functions)
- Keep anonymous blocks simple and focused on a single task
- Use proper indentation to enhance readability of nested blocks
- Include comments to explain complex logic
- Use SET SERVEROUTPUT ON before executing blocks that use DBMS_OUTPUT

Variable Management

- Declare variables in the DECLARE section with appropriate data types
- Initialize variables when declared whenever possible
- Use meaningful variable names following naming conventions:
 1. v_name for regular variables
 2. c_name for constants
 3. g_name for global variables

SQL Integration

- Minimize the number of SQL statements within PL/SQL blocks
- Use bulk operations when processing multiple rows
- Always handle potential exceptions from SQL operations
- Avoid hard-coding values; use bind variables or parameters instead

Exercises

Foundational Exercises

1. Create a simple anonymous PL/SQL block that declares a variable, assigns a value to it, and displays the result using DBMS_OUTPUT.

Applied Exercises

1. Write a PL/SQL block that retrieves an employee's name and salary from the employees table based on an employee ID provided as a substitution variable, then displays the information.

Challenge Problems

1. Create a PL/SQL block that creates a new table called PROD by copying the structure (but not the data) of an existing products table using dynamic SQL.

Quick Reference Cheatsheet

Essential PL/SQL Syntax

```
DECLARE
  -- Variable declarations
  v_variable_name data_type [:= initial_value];
BEGIN
  -- Executable statements
  SELECT column INTO variable FROM table WHERE condition;
  -- Procedural logic
EXCEPTION
  WHEN exception_name THEN
    -- Exception handling code
END;
/
```

Key Commands

- `SET SERVEROUTPUT ON` - Enables output display from `DBMS_OUTPUT`
- `DBMS_OUTPUT.PUT_LINE(text)` - Displays text and moves to next line
- `DBMS_OUTPUT.PUT(text)` - Displays text on current line
- `/` - Executes the current SQL buffer content

Common Error Handling

- `NO_DATA_FOUND` - `SELECT INTO` returns no rows
- `TOO_MANY_ROWS` - `SELECT INTO` returns multiple rows
- `INVALID_NUMBER` - Conversion of string to number fails
- `ZERO_DIVIDE` - Attempt to divide by zero

Block Structure Summary

- **DECLARE:** Optional section for variable, cursor, and exception declarations
 - **BEGIN:** Mandatory section containing executable statements
 - **EXCEPTION:** Optional section for error handling
 - **END:** Mandatory termination of the block
-

Seminar 2: Types of Variables

Overview

This seminar explores the different types of variables available in PL/SQL, including their declaration syntax, scope rules, and practical applications. Variables are fundamental building blocks in PL/SQL programming that allow data manipulation within procedural code. Understanding variable types and their proper usage is critical for writing effective PL/SQL programs.

Key Learning Objectives:

- Differentiate between PL/SQL and non-PL/SQL variables
- Master variable declaration syntax and initialization techniques
- Understand the `%TYPE` attribute and its practical applications
- Learn to work with bind variables and substitution variables
- Apply variables to interact with the Oracle database

Core Concepts

PL/SQL Variables (Local Variables)

PL/SQL variables are declared in the declarative section of a block (or sub-block) and have specific characteristics:

Key Characteristics:

- Used in the executable section (between BEGIN and END)
- Visible in the rest of the block and in nested blocks (except in sub-blocks where their name is redefined)
- Can be passed as parameters to PL/SQL subprograms
- Can be used to store results returned by PL/SQL subprograms
- Follow standard scope rules (visible from point of declaration to end of block)

Variable Declaration Syntax

The basic syntax for declaring PL/SQL variables is:

```
variable_name [CONSTANT] data_type [NOT NULL] [:= | DEFAULT expression]
```

Declaration Elements:

- **Constants:** Must be initialized and cannot change their value later
- **NOT NULL variables:** Must be initialized and cannot later receive NULL values
- **Naming conventions:**
 1. `c_name` for constants
 2. `v_name` for regular variables
 3. `g_name` for global variables

Variable Data Types

PL/SQL supports various data types for variables:

Data Type Category	Data Types	Value Limits	Notes
Character Strings	CHAR, VARCHAR2	Up to 32767 bytes	CHAR = fixed length, VARCHAR2 = variable length
Numeric Values	NUMBER	Precision 1-38, Scale -84 to 127	Most versatile numeric type
Integer Types	BINARYINTEGER, PLSINTEGER	-2147483647 to 2147483647	PLS_INTEGER is faster than NUMBER
Boolean	BOOLEAN	TRUE, FALSE, NULL	Can only be used in PL/SQL, not in SQL statements
Floating Point	BINARYFLOAT, BINARYDOUBLE	IEEE standard formats	For scientific calculations

Non-PL/SQL Variables

These variables are defined outside PL/SQL blocks but can interact with them:

1. **Global/Bind Variables:**
 4. Declared in the host environment (outside PL/SQL)
 5. Remain accessible after PL/SQL execution completes
 6. Cannot be used within procedures, functions, or packages
2. **Substitution Variables:**

7. Used to send values from SQL*Plus environment to SQL commands or PL/SQL blocks
8. Accessed using `&variable_name` or `&&variable_name`
9. Can be read from keyboard using `ACCEPT` command

Syntax Reference

Variable Declaration Examples

```
v_function VARCHAR2(9);
v_totalsal NUMBER(9,2) := 0;
v_startdate DATE := SYSDATE + 7;
c_tax CONSTANT NUMBER(3,2) := 8.25;
v_valid BOOLEAN NOT NULL := TRUE;
v_quantity NUMBER DEFAULT 250;
v_price produce.pret%TYPE := 100;
```

%TYPE Attribute

The `%TYPE` attribute declares a variable according to:

- Another variable's definition: `v_avg_price v_price%TYPE;`
- A table column definition: `v_salary employees.salary%TYPE;`

Bind Variables Syntax

```
-- Declaration in host environment
VARIABLE g_variable_name DATA_TYPE

-- Usage inside PL/SQL
:g_variable_name := v_variable;

-- Displaying outside the block
PRINT g_variable_name
```

Substitution Variables Syntax

```
-- Reading from keyboard
ACCEPT variable_name PROMPT message

-- Using in PL/SQL
&variable_name or &&variable_name

-- Displaying
PROMPT &variable_name
```

Examples

Basic Example: %TYPE Attribute

```
DECLARE
    v_first_name angajati.first_name%TYPE;
    v_last_name angajati.last_name%TYPE;
BEGIN
    SELECT first_name, last_name
    INTO v_first_name, v_last_name
    FROM employees
    WHERE employee_id = 100;

    DBMS_OUTPUT.PUT_LINE('The employee name is: ' || v_first_name || ' ' || v_last_name);
```

```
END;  
/
```

Basic Example: Bind Variables

```
VARIABLE message VARCHAR2(20)  
BEGIN  
    :message := 'testing bind variables';  
END;  
/  
PRINT message
```

Basic Example: Substitution Variables

```
ACCEPT var PROMPT 'Enter an id: '  
DECLARE  
    v_no NUMBER(2);  
BEGIN  
    SELECT COUNT(order_id)  
    INTO v_no  
    FROM orders  
    WHERE customer_id = &var;  
  
    DBMS_OUTPUT.PUT_LINE('The customer made ' || v_no || ' orders');  
END;  
/
```

Advanced Example: Record Type

```
DECLARE  
    TYPE product_type IS RECORD (  
        v_id product_information.product_id%TYPE,  
        v_name product_information.product_name%TYPE,  
        v_price product_information.list_price%TYPE  
    );  
    v_prod product_type;  
BEGIN  
    SELECT product_id, product_name, list_price  
    INTO v_prod  
    FROM product_information  
    WHERE product_id = 2400;  
  
    DBMS_OUTPUT.PUT_LINE('The product ' || v_prod.v_name || ' has the price ' || v_prod.v_price);  
END;  
/
```

Advanced Example: %ROWTYPE

```
DECLARE  
    v_prod product_information%ROWTYPE;  
BEGIN  
    SELECT product_id, product_name, list_price  
    INTO v_prod.product_id, v_prod.product_name, v_prod.list_price  
    FROM product_information  
    WHERE product_id = 2400;  
  
    DBMS_OUTPUT.PUT_LINE('The product ' || v_prod.product_name || ' has the price ' || v_prod.list_price);  
END;  
/
```


Common Patterns & Best Practices

Variable Declaration Best Practices

- Always initialize variables at declaration time when possible
- Use appropriate data types that match the intended usage
- Follow consistent naming conventions for different variable types
- Declare variables close to their first point of use
- Limit variable scope to the smallest necessary block

%TYPE Attribute Usage

- Use %TYPE when declaring variables that will store table column values
- Provides automatic adaptation when table structures change
- Reduces maintenance effort and potential type mismatch errors
- Particularly valuable for columns with complex data types

Bind vs. Substitution Variables

Feature	Bind Variables	Substitution Variables
Scope	Host environment	Session-wide
Persistence	Remain after PL/SQL execution	Remain after PL/SQL execution
Usage in PL/SQL	Accessed with :name	Accessed with &name
Best for	Passing values between PL/SQL blocks	One-time input values
Security	More secure (values not echoed)	Less secure (values may be visible)

Performance Considerations

- Avoid unnecessary conversions between data types
- Use PLS_INTEGER instead of NUMBER for integer calculations (faster)
- Minimize context switching between SQL and PL/SQL engines
- Use bulk operations when processing multiple rows

Exercises

Foundational Exercises

1. Declare variables for an employee's name (VARCHAR2), salary (NUMBER), and hire date (DATE). Initialize the salary to 0 and the hire date to the current date.

Applied Exercises

1. Display the name, total income, and seniority of an employee whose ID is read from the keyboard.
2. Increase the salary of an employee whose ID is read from the keyboard:
 1. Retrieve employee information
 2. Increase salary by 100 in a variable
 3. Update the employee salary

Challenge Problems

1. Create a PL/SQL block that interacts with the Oracle server to create a new table called PROD that has the same structure as an existing products table but contains no data.

Quick Reference Cheatsheet

Variable Declaration Syntax

```
-- Basic declaration
variable_name [CONSTANT] data_type [NOT NULL] [:= | DEFAULT expression]

-- Examples
v_name VARCHAR2(50);
c_tax_rate CONSTANT NUMBER(3,2) := 0.08;
v_is_valid BOOLEAN NOT NULL := TRUE;
```

%TYPE Attribute

```
-- Declare based on table column
v_salary employees.salary%TYPE;

-- Declare based on another variable
v_avg_price v_salary%TYPE;
```

Bind Variables

```
-- Host environment declaration
VARIABLE var_name DATA_TYPE

-- Inside PL/SQL block
:var_name := plsql_variable;

-- Display results
PRINT var_name
```

Substitution Variables

```
-- Read from keyboard
ACCEPT var PROMPT 'Message: '

-- Use in PL/SQL
&var_name

-- Display
PROMPT &var_name
```

Record Types

```
-- User-defined record
TYPE record_name IS RECORD (
    field1 data_type,
    field2 data_type DEFAULT value
);
var_name record_name;

-- %ROWTYPE record
var_name table_name%ROWTYPE;
```

Common Variable Patterns

- **Constants:** Always use for values that shouldn't change (tax rates, conversion factors)
 - **NOT NULL:** Use when a variable must always have a value
 - **%TYPE:** Best practice for variables storing database values
 - **Bind Variables:** Use for passing values between SQL*Plus and PL/SQL
 - **Substitution Variables:** Use for one-time user input values
-

Seminar 3: Fundamental Programming Structures

Overview

This seminar covers the essential control structures in PL/SQL that enable conditional execution and repetition of code blocks. These structures form the backbone of procedural programming in PL/SQL, allowing developers to implement complex business logic that goes beyond simple SQL statements. Mastery of these structures is crucial for writing effective PL/SQL programs.

Key Learning Objectives:

- Understand and implement alternative (conditional) structures (IF, CASE)
- Master repetitive (looping) structures (LOOP, WHILE, FOR)
- Learn to properly nest control structures
- Apply these structures to solve real-world database problems
- Understand best practices for using control structures efficiently

Core Concepts

Alternative Structures

Alternative structures allow conditional execution of code based on specified conditions.

IF-THEN-ELSE Structure

The basic IF structure provides conditional execution:

```
IF condition THEN
    sequence_of_statements
END IF;
```

Extended with ELSE clause:

```
IF condition THEN
    sequence_of_statements
ELSE
    sequence_of_statements
END IF;
```

Nested IF statements can be simplified using ELSIF:

```
IF condition1 THEN
    statements
ELSIF condition2 THEN
    statements
ELSIF condition3 THEN
    statements
```

```
...
ELSE
    statements
END IF;
```

CASE Structure

The CASE structure provides an alternative to complex IF-ELSIF chains.

CASE Expression (returns a value):

```
variable := CASE [selector]
    WHEN expression1 THEN result1
    WHEN expression2 THEN result2
    ...
    [ELSE resultN]
END;
```

CASE Statement (executes actions):

```
CASE [selector]
    WHEN expression1 THEN action1;
    WHEN expression2 THEN action2;
    ...
    [ELSE actionN;]
END CASE;
```

Repetitive Structures

Repetitive structures enable the execution of code blocks multiple times.

Basic LOOP Structure

The simplest loop structure:

```
LOOP
    sequence_of_statements
    EXIT [WHEN condition];
END LOOP;
```

WHILE Loop Structure

Executes as long as a condition is true:

```
WHILE condition LOOP
    sequence_of_statements
    [EXIT WHEN condition];
END LOOP;
```

FOR Loop Structure

Iterates over a specified range:

```
FOR counter IN [REVERSE] lower_bound..upper_bound LOOP
    sequence_of_statements
    [EXIT WHEN condition];
END LOOP;
```

Syntax Reference

IF Statement Syntax Variations

Structure	Syntax
Simple IF	IF condition THEN statements END IF;
IF-ELSE	IF condition THEN statements ELSE statements END IF;
IF-ELSIF	IF cond1 THEN stmts ELSIF cond2 THEN stmts ... ELSE stmts END IF;

CASE Statement Syntax

Type	Syntax
CASE Expression	variable := CASE [selector] WHEN expr THEN result ... [ELSE result] END;
CASE Statement	CASE [selector] WHEN expr THEN action; ... [ELSE action;] END CASE;

Loop Structures Syntax

Loop Type	Syntax
Basic LOOP	LOOP statements EXIT [WHEN condition]; END LOOP;
WHILE LOOP	WHILE condition LOOP statements [EXIT WHEN condition]; END LOOP;
FOR LOOP	FOR counter IN [REVERSE] min..max LOOP statements [EXIT WHEN condition]; END LOOP;

Examples

Basic Example: IF-THEN-ELSE

```
DECLARE
    grade NUMBER := 7;
BEGIN
    IF grade < 5 THEN
        DBMS_OUTPUT.PUT_LINE('Sorry, the candidate is rejected!!');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Congratulations, you are admitted!');
    END IF;
END;
```

Basic Example: CASE Expression

```
DECLARE
    v_list products.list_price%TYPE;
    v_percent NUMBER(4,2);
BEGIN
    SELECT list_price INTO v_list
    FROM products
    WHERE product_id = &p;
```

```

DBMS_OUTPUT.PUT_LINE('The initial list_price is: ' || v_list);

v_percent := CASE
    WHEN v_list < 500 THEN 2
    WHEN v_list BETWEEN 500 AND 1000 THEN 1.5
    ELSE 1.25
END;

v_list := v_list * v_percent;
DBMS_OUTPUT.PUT_LINE('The final price is ' || v_list);
END;
/

```

Basic Example: CASE Statement

```

DECLARE
    v_list products.list_price%TYPE;
BEGIN
    SELECT list_price INTO v_list
    FROM products
    WHERE product_id = &p;

    DBMS_OUTPUT.PUT_LINE('The initial list_price is: ' || v_list);

    CASE
        WHEN v_list < 500 THEN v_list := 2 * v_list;
        WHEN v_list BETWEEN 500 AND 1000 THEN v_list := 1.5 * v_list;
        ELSE v_list := 1.25 * v_list;
    END CASE;

    DBMS_OUTPUT.PUT_LINE('The final price is ' || v_list);
END;
/

```

Basic Example: LOOP Structure

```

DECLARE
    v_nr NUMBER(2) := 10;
    i NUMBER(2) := 1;
BEGIN
    LOOP
        v_nr := v_nr - i;
        i := i + 1;
        EXIT WHEN v_nr < 0;
        DBMS_OUTPUT.PUT_LINE(v_nr);
    END LOOP;
END;
/

```

Basic Example: WHILE Loop

```

DECLARE
    v_nr NUMBER(2) := 10;
    i NUMBER(2) := 1;
BEGIN
    WHILE v_nr > 0 LOOP
        v_nr := v_nr - i;
        i := i + 1;
        DBMS_OUTPUT.PUT_LINE(v_nr);
    END LOOP;
END;
/

```

Basic Example: FOR Loop

```
DECLARE
    v_nr NUMBER(2) := 10;
    i NUMBER(2);
BEGIN
    FOR i IN 1..10 LOOP
        v_nr := v_nr - i;
        EXIT WHEN v_nr < 0;
        DBMS_OUTPUT.PUT_LINE(v_nr);
    END LOOP;
END;
/
```

Advanced Example: Employee Salary Update

```
DECLARE
    v_emp_id NUMBER := &employee_id;
    v_name VARCHAR2(50);
    v_salary NUMBER;
    v_orders NUMBER;
BEGIN
    -- Retrieve employee information
    SELECT first_name || ' ' || last_name, salary
    INTO v_name, v_salary
    FROM employees
    WHERE employee_id = v_emp_id;

    DBMS_OUTPUT.PUT_LINE('Employee ' || v_name || ' has initial salary: ' || v_salary);

    -- Count orders handled by employee
    SELECT COUNT(*)
    INTO v_orders
    FROM orders
    WHERE sales_rep_id = v_emp_id;

    DBMS_OUTPUT.PUT_LINE('Employee handled ' || v_orders || ' orders');

    -- Update salary based on order count
    IF v_orders BETWEEN 4 AND 8 THEN
        UPDATE employees SET salary = salary * 1.1 WHERE employee_id = v_emp_id;
        DBMS_OUTPUT.PUT_LINE('Salary increased by 10%');
    ELSIF v_orders > 8 THEN
        UPDATE employees SET salary = salary * 1.2 WHERE employee_id = v_emp_id;
        DBMS_OUTPUT.PUT_LINE('Salary increased by 20%');
    ELSE
        DBMS_OUTPUT.PUT_LINE('No salary change');
    END IF;

    -- Display new salary
    SELECT salary INTO v_salary FROM employees WHERE employee_id = v_emp_id;
    DBMS_OUTPUT.PUT_LINE('New salary: ' || v_salary);
END;
/
```

Common Patterns & Best Practices

Conditional Structures Best Practices

- Use CASE instead of complex IF-ELSIF chains when possible (more readable)
- Always include an ELSE clause in conditional structures to handle unexpected cases
- Keep conditions simple and avoid deeply nested conditional structures
- For NULL handling, remember that comparisons with NULL yield NULL, not TRUE/FALSE

- Test edge cases thoroughly (boundary values, NULL values)

Loop Structures Best Practices

- Prefer FOR loops when the number of iterations is known
- Use WHILE loops when iteration depends on a condition
- Use basic LOOP with EXIT WHEN for more complex iteration logic
- Avoid infinite loops by ensuring exit conditions are always reachable
- Use EXIT WHEN rather than IF-THEN-EXIT for cleaner code
- Consider performance implications of nested loops

General Control Structure Guidelines

- Document the purpose of complex control structures
- Keep individual code blocks focused on a single task
- Consider using functions/procedures for complex logic that's reused
- Test boundary conditions thoroughly
- Use meaningful variable names that reflect their purpose in the control structure

Exercises

Foundational Exercises

1. Create a PL/SQL block that displays numbers from 1 to 10 using each of the three loop structures (LOOP, WHILE, FOR).

Applied Exercises

1. In a PL/SQL block, update an employee's salary based on number of handled orders:
 1. 4-8 orders → salary + 10%
 2. More than 8 → salary + 20%
 3. Otherwise → no change
 4. Display the name and initial salary
 5. Identify and display the number of orders for the employee
 6. Perform the update
 7. Display the new salary received by the employee
2. In a PL/SQL block, iterate through all employees with employee_id from 100 to 110, displaying their name and salary (exemplify using all 3 repetitive structures).

Challenge Problems

1. Write a PL/SQL block that iterates through all employees, displaying their name and salary using all three repetitive structures. Handle the case where some employee IDs might not exist.
2. Implement a Fibonacci sequence generator using each of the three loop structures, displaying the first 10 numbers in the sequence.

Quick Reference Cheatsheet

Conditional Structures

```
-- Simple IF
IF condition THEN
    statements
END IF;

-- IF-ELSE
```



```

IF condition THEN
    statements
ELSE
    statements
END IF;

-- IF-ELSIF
IF cond1 THEN
    stmts
ELSIF cond2 THEN
    stmts
ELS
    stmts
END IF;

-- CASE Expression
var := CASE
    WHEN expr1 THEN result1
    WHEN expr2 THEN result2
    ELSE result
END;

-- CASE Statement
CASE
    WHEN expr1 THEN stmts;
    WHEN expr2 THEN stmts;
    ELSE stmts;
END CASE;

```

Loop Structures

```

-- Basic LOOP
LOOP
    statements
    EXIT WHEN condition;
END LOOP;

-- WHILE LOOP
WHILE condition LOOP
    statements
    EXIT WHEN condition;
END LOOP;

-- FOR LOOP
FOR counter IN min..max LOOP
    statements
    EXIT WHEN condition;
END LOOP;

```

Common Patterns

- **EXIT vs RETURN:** EXIT leaves a loop, RETURN leaves a subprogram
- **NULL handling:** Remember that NULL = NULL returns NULL, not TRUE
- **Loop counters:** FOR loop counters don't need declaration
- **Infinite loops:** Always ensure exit conditions are reachable
- **Loop nesting:** Limit nesting depth for readability

Performance Tips

- FOR loops are generally more efficient than WHILE or basic LOOP
- Minimize SQL statements inside loops (use bulk operations when possible)
- Avoid unnecessary computations inside loop bodies
- Consider using CONTINUE for complex loop conditions

Seminar 4: Composite Data Types

Overview

This seminar explores composite data types in PL/SQL, focusing primarily on RECORD types. Composite data types allow grouping related data elements into a single logical unit, providing more structured and organized data handling. Understanding these types is essential for working with complex data structures in PL/SQL applications.

Key Learning Objectives:

- Understand the concept and purpose of composite data types
- Master the declaration and usage of RECORD types
- Learn to use %ROWTYPE for table-based records
- Apply composite data types to solve real-world database problems
- Understand best practices for using composite data types efficiently

Core Concepts

RECORD Data Type

A RECORD is a composite data type that groups logically related data elements into a single structure.

Key Characteristics:

- It's a group of data logically correlated
- It can be handled as a single unit
- Each element of the structure (field) has a name and a data type
- The fields of a record are referenced using dot notation: `record_name.field_name`

RECORD Declaration Syntax

Records can be declared in two main ways:

1. User-Defined Record Type:

```
TYPE record_type_name IS RECORD (  
    field1 data_type [:= | DEFAULT expression],  
    field2 data_type [:= | DEFAULT expression],  
    ...  
);  
variable_name record_type_name;
```

1. %ROWTYPE Attribute:

```
variable_name table_name%ROWTYPE;
```

%ROWTYPE vs User-Defined Records

Feature	%ROWTYPE	User-Defined Records
Definition	Based directly on table structure	Explicitly defined by developer
Flexibility	Limited to table columns	Can include any combination of fields
Maintenance	Automatically adapts to table changes	Requires manual updates when structure

		changes
Performance	Slightly less efficient	More efficient for specific needs
Usage	Simple declaration	More verbose declaration

Syntax Reference

User-Defined Record Syntax

```

TYPE record_name IS RECORD (
    field_name1 data_type [NOT NULL] [:= | DEFAULT expression],
    field_name2 data_type [NOT NULL] [:= | DEFAULT expression],
    ...
);
variable_name record_name;

```

%ROWTYPE Syntax

```

variable_name table_name%ROWTYPE;

```

Record Field Access

```

-- Assign values to record fields
record_name.field_name := value;

-- Access record field values
value := record_name.field_name;

```

Examples

Basic Example: User-Defined Record

```

DECLARE
    TYPE product_type IS RECORD (
        v_cod product_information.product_id%TYPE NOT NULL := 3000,
        v_name product_information.product_name%TYPE,
        v_min_price product_information.min_price%TYPE
    );
    vrec_prod product_type;
BEGIN
    SELECT product_id, product_name, min_price
    INTO vrec_prod
    FROM product_information
    WHERE product_id = 3133;

    DBMS_OUTPUT.PUT_LINE('Product ' || vrec_prod.v_name || ' has a minimum price of: ' ||
vrec_prod.v_min_price);
END;
/

```

Basic Example: %ROWTYPE

```

DECLARE
    v_prod product_information%ROWTYPE;
BEGIN
    SELECT product_id, product_name, list_price
    INTO v_prod.product_id, v_prod.product_name, v_prod.list_price
    FROM product_information
    WHERE product_id = 2400;

```

```

DBMS_OUTPUT.PUT_LINE('The product ' || v_prod.product_name || ' has the price ' || v_prod.list_price);
END;
/

```

Advanced Example: Processing Multiple Records

```

DECLARE
    TYPE product_type IS RECORD (
        v_id product_information.product_id%TYPE,
        v_name product_information.product_name%TYPE,
        v_price product_information.list_price%TYPE
    );
    v_prod product_type;

    CURSOR c_products IS
        SELECT product_id, product_name, list_price
        FROM product_information
        WHERE list_price IS NOT NULL
        ORDER BY list_price DESC;
BEGIN
    DBMS_OUTPUT.PUT_LINE('The most expensive products are:');

    OPEN c_products;
    LOOP
        FETCH c_products INTO v_prod;
        EXIT WHEN c_products%NOTFOUND OR c_products%ROWCOUNT = 4;

        DBMS_OUTPUT.PUT_LINE('The product ' || v_prod.v_id || ' has the price: ' || v_prod.v_price);
    END LOOP;
    CLOSE c_products;
END;
/

```

Advanced Example: Nested Records

```

DECLARE
    TYPE address_type IS RECORD (
        street VARCHAR2(50),
        city VARCHAR2(30),
        state VARCHAR2(20),
        zip_code VARCHAR2(10)
    );

    TYPE employee_type IS RECORD (
        emp_id NUMBER,
        emp_name VARCHAR2(50),
        emp_address address_type,
        salary NUMBER
    );

    v_emp employee_type;
BEGIN
    -- Initialize nested record
    v_emp.emp_id := 101;
    v_emp.emp_name := 'John Smith';
    v_emp.emp_address.street := '123 Main St';
    v_emp.emp_address.city := 'New York';
    v_emp.emp_address.state := 'NY';
    v_emp.emp_address.zip_code := '10001';
    v_emp.salary := 50000;

    DBMS_OUTPUT.PUT_LINE('Employee: ' || v_emp.emp_name);
    DBMS_OUTPUT.PUT_LINE('Address: ' || v_emp.emp_address.street || ', ' ||
        v_emp.emp_address.city || ', ' ||

```

```
        v_emp.emp_address.state || ' ' ||  
        v_emp.emp_address.zip_code);  
DBMS_OUTPUT.PUT_LINE('Salary: $' || v_emp.salary);  
END;  
/
```

Common Patterns & Best Practices

Record Type Selection Guidelines

- Use **%ROWTYPE** when you need all or most columns from a single table
- Use **user-defined records** when:
 1. You need columns from multiple tables
 2. You need only specific columns from a table
 3. You need to include calculated fields
 4. You need to define custom default values

Performance Considerations

- Avoid selecting unnecessary columns when using records
- For large tables, consider using user-defined records with only needed fields
- When processing multiple rows, consider bulk operations with collections
- Be careful with %ROWTYPE when table structures change frequently

Best Practices for Record Usage

- Use meaningful field names that clearly indicate their purpose
- Initialize records or their fields when declared
- Use dot notation consistently for field access
- Document the purpose of complex record structures
- Consider creating record types in packages for reuse across multiple programs

Common Pitfalls to Avoid

- Trying to assign an entire record to another record of different structure
- Assuming %ROWTYPE will include columns added after the PL/SQL unit is compiled
- Using records for simple data that could be handled with scalar variables
- Forgetting that %ROWTYPE records cannot be used in SQL statements directly

Exercises

Foundational Exercises

1. Using a RECORD data type of the same type as a row in the Departments table, display the name of each department with the ID 10, 20, 30, 40, 50.

Applied Exercises

1. Create a user-defined record type to store customer information (ID, name, email, phone) and order information (order ID, order date, total amount). Write a PL/SQL block that retrieves and displays this combined information for a specific customer.

Challenge Problems

1. Implement a PL/SQL block that uses nested records to represent a hierarchical structure of departments, managers, and employees. Display the organization structure with proper indentation.

Quick Reference Cheatsheet

Record Declaration

```
-- User-defined record
TYPE record_type IS RECORD (
  field1 data_type [DEFAULT expr],
  field2 data_type [DEFAULT expr],
  ...
);
var_name record_type;

-- %ROWTYPE record
var_name table_name%ROWTYPE;
```

Record Operations

```
-- Assign values
var_name.field := value;

-- Select into record
SELECT col1, col2 INTO var_name.field1, var_name.field2 FROM table;

-- %ROWTYPE select
SELECT * INTO var_name FROM table WHERE condition;
```

Key Differences

Feature	%ROWTYPE	User-Defined Records
Declaration	Simple (one line)	More verbose
Structure	Matches table exactly	Custom structure
Flexibility	Limited	High
Maintenance	Automatic with table changes	Manual updates needed
Best For	Simple table access	Complex data structures

Common Use Cases

- Mapping query results to structured data
 - Passing multiple values between procedures/functions
 - Creating temporary data structures
 - Representing business entities with multiple attributes
 - Handling complex join results
-

Seminar 5: Explicit Cursor

Overview

This seminar covers explicit cursors in PL/SQL, which provide fine-grained control over the processing of query result sets. While implicit cursors are automatically managed by PL/SQL for single-row queries, explicit cursors are essential for processing multiple rows with custom logic. Understanding explicit cursors is crucial for developing efficient database applications that require row-by-row processing.

Key Learning Objectives:

- Understand the cursor concept and its role in PL/SQL
- Master the declaration and management of explicit cursors
- Learn different cursor iteration techniques (OPEN-LOOP-FETCH-CLOSE, FOR LOOP)
- Work with parameterized cursors for flexible querying
- Apply the FOR UPDATE clause for row locking during updates

Core Concepts

Cursor Fundamentals

A cursor is a pointer to a memory area called the context area, where query results are stored. The cursor mechanism allows transitioning from set-based SQL operations to row-level processing.

Cursor Types:

- **Implicit Cursors:** Automatically created and managed by PL/SQL for all DML statements
- **Explicit Cursors:** Declared and managed by the programmer, primarily for SELECT statements

Explicit Cursor Processing Steps

Processing an explicit cursor involves four key steps:

1. **Declare** the cursor with a name and associated query
2. **Open** the cursor, which executes the query and creates the active set
3. **Fetch** data from the cursor into variables
4. **Close** the cursor to release resources

Cursor Attributes

Explicit cursors provide several attributes to check their state:

Attribute	Returns	Description
%ISOPEN	BOOLEAN	TRUE if cursor is open
%NOTFOUND	BOOLEAN	TRUE if last FETCH returned no row
%FOUND	BOOLEAN	TRUE if last FETCH returned a row (opposite of %NOTFOUND)
%ROWCOUNT	INTEGER	Number of rows fetched so far

Parameterized Cursors

Cursors can accept parameters to make them more flexible and reusable:

```
CURSOR cursor_name(parameter1 data_type, ...) IS
    SELECT ... WHERE condition = parameter1;
```

FOR UPDATE Clause

Used to lock rows when the cursor is opened, preventing other sessions from modifying them:

```
CURSOR cursor_name IS
    SELECT ... FROM ... FOR UPDATE [OF column_list] [NOWAIT | WAIT n];
```

WHERE CURRENT OF Clause

Allows modifying the current row of a cursor without specifying a WHERE condition:

```
UPDATE table SET column = value WHERE CURRENT OF cursor_name;
```

Syntax Reference

Explicit Cursor Declaration

```
DECLARE
    CURSOR cursor_name IS
        SELECT column_list FROM table WHERE condition;
    -- OR with parameters
    CURSOR cursor_name(parameter data_type) IS
        SELECT column_list FROM table WHERE condition = parameter;
```

Cursor Processing

```
OPEN cursor_name;
LOOP
    FETCH cursor_name INTO variables;
    EXIT WHEN cursor_name%NOTFOUND;
    -- Process row
END LOOP;
CLOSE cursor_name;
```

FOR LOOP Cursor Processing

```
FOR record_name IN cursor_name LOOP
    -- Process row using record_name.field
END LOOP;
-- OR with inline cursor
FOR record_name IN (SELECT ...) LOOP
    -- Process row
END LOOP;
```

Parameterized Cursor Usage

```
OPEN cursor_name(parameter_value);
LOOP
    FETCH cursor_name INTO variables;
    EXIT WHEN cursor_name%NOTFOUND;
    -- Process row
END LOOP;
CLOSE cursor_name;
```


FOR UPDATE Syntax

```
CURSOR cursor_name IS
  SELECT ... FROM ... FOR UPDATE [OF column] [NOWAIT | WAIT n];
```

Examples

Basic Example: OPEN-LOOP-FETCH-CLOSE

```
DECLARE
  CURSOR emp_cursor IS
    SELECT employee_id, first_name, last_name, salary
    FROM employees
    WHERE department_id = 60;
  emp_rec emp_cursor%ROWTYPE;
BEGIN
  DBMS_OUTPUT.PUT_LINE('Employee salaries in department 60');
  OPEN emp_cursor;
  LOOP
    FETCH emp_cursor INTO emp_rec;
    EXIT WHEN emp_cursor%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE('Employee ' || emp_rec.first_name || ' ' ||
                        emp_rec.last_name || ' has salary: ' || emp_rec.salary);
  END LOOP;
  CLOSE emp_cursor;
END;
/
```

Basic Example: FOR LOOP with Named Cursor

```
DECLARE
  CURSOR emp_cursor IS
    SELECT employee_id, first_name, last_name, salary
    FROM employees
    WHERE department_id = 60;
BEGIN
  DBMS_OUTPUT.PUT_LINE('Employee salaries in department 60');
  FOR emp_rec IN emp_cursor LOOP
    DBMS_OUTPUT.PUT_LINE('Employee ' || emp_rec.first_name || ' ' ||
                        emp_rec.last_name || ' has salary: ' || emp_rec.salary);
  END LOOP;
END;
/
```

Basic Example: Inline Cursor in FOR LOOP

```
BEGIN
  DBMS_OUTPUT.PUT_LINE('Total salaries by department:');
  FOR dep_rec IN (
    SELECT d.department_id, d.department_name, SUM(e.salary) total_salary
    FROM departments d
    JOIN employees e ON d.department_id = e.department_id
    GROUP BY d.department_id, d.department_name
  ) LOOP
    DBMS_OUTPUT.PUT_LINE('Department ' || dep_rec.department_name ||
                        ' pays total salaries of: ' || dep_rec.total_salary || ' RON');
  END LOOP;
END;
/
```

Advanced Example: Parameterized Cursor

```
DECLARE
    CURSOR c_prod(p_val NUMBER) IS
        SELECT p.product_id prod, SUM(quantity) total_quantity
        FROM product_information p
        JOIN order_items o ON p.product_id = o.product_id
        GROUP BY p.product_id
        HAVING SUM(quantity) > p_val
        ORDER BY total_quantity DESC;

    v_val NUMBER(5);
    rec_prod c_prod%ROWTYPE;
BEGIN
    v_val := 50;
    DBMS_OUTPUT.PUT_LINE('The products that have a total ordered quantity larger than ' || v_val);

    IF NOT c_prod%ISOPEN THEN
        OPEN c_prod(v_val);
    END IF;

    LOOP
        FETCH c_prod INTO rec_prod;
        EXIT WHEN c_prod%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE(rec_prod.total_quantity || ' pieces of product ' ||
                               rec_prod.prod || ' were ordered');
    END LOOP;

    CLOSE c_prod;
END;
/
```

Advanced Example: FOR UPDATE with WHERE CURRENT OF

```
DECLARE
    CURSOR c_update IS
        SELECT *
        FROM orders
        FOR UPDATE OF order_date NOWAIT;
BEGIN
    FOR rec IN c_update LOOP
        DBMS_OUTPUT.PUT_LINE('The order ' || rec.order_id ||
                               ' has the date: ' || rec.order_date);

        UPDATE orders
        SET order_date = order_date - 7
        WHERE CURRENT OF c_update;

        DBMS_OUTPUT.PUT_LINE('The order ' || rec.order_id ||
                               ' has the date: ' || rec.order_date);
    END LOOP;
END;
/
```

Common Patterns & Best Practices

Cursor Selection Guidelines

- Use **implicit cursors** for single-row queries
- Use **explicit cursors** when:
 1. Processing multiple rows with custom logic
 2. Needing to control the fetch process
 3. Requiring cursor attributes for conditional processing

4. Working with parameterized queries

Cursor Processing Best Practices

- Always close cursors after use to free resources
- Use `%NOTFOUND` for loop termination (not `%FOUND`)
- For simple processing, prefer FOR loops over explicit OPEN/FETCH/CLOSE
- Use parameterized cursors for reusable query logic
- When using FOR UPDATE, specify only necessary columns in OF clause
- Consider NOWAIT vs WAIT based on application requirements

Performance Considerations

- Avoid fetching more rows than necessary
- Use WHERE clauses to filter at the database level
- Consider bulk operations for large result sets
- Close cursors as soon as processing is complete
- For frequently executed queries, consider keeping cursors open

Common Pitfalls to Avoid

- Forgetting to close cursors (resource leaks)
- Using `%FOUND` before the first FETCH (returns NULL)
- Assuming `%NOTFOUND` is FALSE before the first FETCH (it's NULL)
- Fetching into variables with incompatible data types
- Using WHERE CURRENT OF without FOR UPDATE clause

Exercises

Foundational Exercises

1. Display the first 3 orders that contain the highest number of ordered products. The records should be sorted in descending order based on the number of products.

Applied Exercises

1. Write a PL/SQL block that displays for each department (id, name) the total salary paid to employees.
2. Write a PL/SQL block that displays for each employee (id, name) details about the orders handled by them (order id, date, order mode).

Challenge Problems

1. Create a PL/SQL block that uses a parameterized cursor to display customers and their orders. The cursor should accept a customer ID and return all orders for that customer.

Quick Reference Cheatsheet

Cursor Processing Methods

```
-- Method 1: Explicit OPEN-LOOP-FETCH-CLOSE
OPEN cursor;
LOOP
    FETCH cursor INTO variables;
    EXIT WHEN cursor%NOTFOUND;
    -- Process row
END LOOP;
CLOSE cursor;
```

```
-- Method 2: FOR LOOP with named cursor
FOR rec IN cursor_name LOOP
    -- Process rec.field
END LOOP;

-- Method 3: FOR LOOP with inline cursor
FOR rec IN (SELECT ...) LOOP
    -- Process rec.field
END LOOP;
```

Cursor Attributes

Attribute	Before OPEN	After OPEN, Before FETCH	After FETCH (Success)	After FETCH (Fail)
%ISOPEN	FALSE	TRUE	TRUE	TRUE
%FOUND	NULL	NULL	TRUE	FALSE
%NOTFOUND	NULL	NULL	FALSE	TRUE
%ROWCOUNT	NULL	0	>0	Same as previous

Parameterized Cursor

```
CURSOR cursor_name(param data_type) IS
    SELECT ... WHERE column = param;

OPEN cursor_name(value);
```

FOR UPDATE Syntax

```
CURSOR cursor_name IS
    SELECT ... FROM ... FOR UPDATE [OF column] [NOWAIT | WAIT n];

-- Then use in UPDATE:
UPDATE table SET ... WHERE CURRENT OF cursor_name;
```

Key Recommendations

- Use FOR loops for simple cursor processing
- Use explicit OPEN/FETCH/CLOSE when you need more control
- Always close cursors to prevent resource leaks
- Parameterized cursors improve code reuse
- FOR UPDATE locks rows for the duration of the transaction

Seminar 6: Exception Handling

Overview

This seminar covers exception handling in PL/SQL, a critical mechanism for managing errors and abnormal conditions in database applications. Proper exception handling transforms brittle code into robust applications that can gracefully handle unexpected situations. Understanding exception types, propagation, and handling techniques is essential for developing reliable PL/SQL programs.

Key Learning Objectives:

- Understand the concept and importance of exception handling

- Differentiate between predefined, non-predefined, and user-defined exceptions
- Master the syntax and structure of exception handling blocks
- Learn to use SQLCODE and SQLERRM for error diagnostics
- Apply best practices for effective exception handling

Core Concepts

Exception Fundamentals

An exception is a PL/SQL identifier associated with an abnormal condition that occurs during execution. When an exception occurs, normal execution stops, and control transfers to an exception handler.

Key Characteristics:

- Exceptions can be raised automatically by Oracle or explicitly by the programmer
- Unhandled exceptions cause program termination
- Exception handlers provide a way to manage errors gracefully
- Exceptions can propagate to outer blocks if not handled

Types of Exceptions

PL/SQL supports three main types of exceptions:

Type	Description	Declaration	Raising	Handling
Predefined Oracle Exceptions	Associated with common Oracle errors (e.g., NODATAFOUND)	Not required	Automatic by Oracle	Required in EXCEPTION section
Non-Predefined Oracle Exceptions	Oracle errors without predefined names (e.g., ORA-01400)	Required with EXCEPTION_INIT	Automatic by Oracle	Required in EXCEPTION section
User-Defined Exceptions	Custom exceptions for application-specific conditions	Required with EXCEPTION	Explicit with RAISE	Required in EXCEPTION section

Exception Handling Structure

The EXCEPTION section has the following structure:

```

EXCEPTION
  WHEN exception1 [OR exception2...] THEN
    statements;
  [WHEN exception3 [OR exception4...] THEN
    statements;]
  [WHEN OTHERS THEN
    statements;]
```

Exception Propagation

When an exception occurs and isn't handled in the current block, it propagates to the enclosing block. This continues until either:

- A handler is found, or
- The exception reaches the outermost block, causing program termination

Syntax Reference

Predefined Exception Handling

```
EXCEPTION
  WHEN predefined_exception THEN
    -- Handle error
  WHEN OTHERS THEN
    -- Handle all other errors
END;
```

Non-Predefined Exception Handling

```
DECLARE
  exception_name EXCEPTION;
  PRAGMA EXCEPTION_INIT(exception_name, error_code);
BEGIN
  -- Code that might raise error
EXCEPTION
  WHEN exception_name THEN
    -- Handle error
END;
```

User-Defined Exception Handling

```
DECLARE
  exception_name EXCEPTION;
BEGIN
  -- Code that might raise error
  IF condition THEN
    RAISE exception_name;
  END IF;
EXCEPTION
  WHEN exception_name THEN
    -- Handle error
END;
```

RAISEAPPLICATIONERROR

```
RAISE_APPLICATION_ERROR(error_number, error_message);
-- error_number must be between -20999 and -20000
```

SQLCODE and SQLERRM

```
DECLARE
  code NUMBER;
  message VARCHAR2(255);
BEGIN
  -- Code that might raise error
EXCEPTION
  WHEN OTHERS THEN
    code := SQLCODE;
    message := SQLERRM;
    -- Handle error using code and message
END;
```

Examples

Basic Example: Predefined Exception

```
SET SERVEROUTPUT ON
DECLARE
    v_name VARCHAR2(20);
BEGIN
    SELECT name INTO v_name
    FROM employees
    WHERE employee_id = 10;

    DBMS_OUTPUT.PUT_LINE(v_name);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Employee with this ID does not exist!');
END;
/
```

Basic Example: Non-Predefined Exception

```
CREATE TABLE errors (
    user_name VARCHAR2(40),
    error_date DATE,
    error_code NUMBER(10),
    error_message VARCHAR2(255)
);

DECLARE
    delete_exception EXCEPTION;
    PRAGMA EXCEPTION_INIT(delete_exception, -2292);
    code NUMBER;
    message VARCHAR2(255);
BEGIN
    DELETE FROM products;
EXCEPTION
    WHEN delete_exception THEN
        DBMS_OUTPUT.PUT_LINE('You cannot delete the product');
        DBMS_OUTPUT.PUT_LINE('Orders are assigned to it');
        code := SQLCODE;
        message := SQLERRM;
        INSERT INTO errors VALUES(USER, SYSDATE, code, message);
END;
/
```

Basic Example: User-Defined Exception

```
DECLARE
    invalid_prod EXCEPTION;
BEGIN
    UPDATE products
    SET product_name = 'Laptop ABC'
    WHERE product_id = 3;

    IF SQL%NOTFOUND THEN
        RAISE invalid_prod;
    END IF;
EXCEPTION
    WHEN invalid_prod THEN
        DBMS_OUTPUT.PUT_LINE('Product does not exist');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error occurred. Cannot update');
END;
/
```

Advanced Example: RAISEAPPLICATIONERROR

```
DECLARE
    e_exc1 EXCEPTION;
    PRAGMA EXCEPTION_INIT(e_exc1, -20600);
BEGIN
    IF TO_NUMBER(TO_CHAR(SYSDATE, 'HH24')) <= 17 THEN
        RAISE_APPLICATION_ERROR(-20600, 'Operation not yet allowed');
    END IF;
EXCEPTION
    WHEN e_exc1 THEN
        DBMS_OUTPUT.PUT_LINE('It is ' || TO_CHAR(SYSDATE, 'HH24'));
        DBMS_OUTPUT.PUT_LINE('Operation not yet allowed');
        DBMS_OUTPUT.PUT_LINE(SQLERRM);
END;
/
```

Advanced Example: Exception Propagation

```
<<outer_block>>
DECLARE
    exc EXCEPTION;
    var NUMBER;
BEGIN
    <<inner_block>>
    BEGIN
        DBMS_OUTPUT.PUT_LINE('Message in inner block');
        RAISE exc;
    END inner_block;

    var := 6/0;
    DBMS_OUTPUT.PUT_LINE('Message in outer block');
EXCEPTION
    WHEN exc THEN
        DBMS_OUTPUT.PUT_LINE('An exception was handled!');
    WHEN ZERO_DIVIDE THEN
        DBMS_OUTPUT.PUT_LINE(SQLERRM);
END outer_block;
/
```

Common Patterns & Best Practices

Exception Handling Best Practices

- Always include an exception handler in top-level blocks
- Handle specific exceptions before using WHEN OTHERS
- Log detailed error information (SQLCODE, SQLERRM)
- Avoid empty exception handlers (at minimum, log the error)
- Use RAISE to re-raise exceptions after partial handling
- Consider custom error messages for user-facing applications

Predefined Exception Guidelines

- NO_DATA_FOUND: Handle cases where expected data doesn't exist
- TOO_MANY_ROWS: Handle cases where single-row query returns multiple rows
- ZERO_DIVIDE: Prevent division by zero errors
- INVALID_NUMBER: Handle conversion errors from string to number
- DUP_VAL_ON_INDEX: Handle unique constraint violations

Error Logging Strategies

- Create a dedicated error logging table
- Include timestamp, user, error code, and message
- Consider including call stack information
- Balance detail with performance impact
- Implement a reusable error logging procedure

Common Pitfalls to Avoid

- Using WHEN OTHERS without re-raising or logging
- Hiding important error information from users
- Overusing exception handling for normal program flow
- Ignoring exceptions by using empty handlers
- Not considering exception propagation in nested blocks

Exercises

Foundational Exercises

1. Write a PL/SQL block that attempts to select a product with ID 13. Handle the case where the product doesn't exist.

Applied Exercises

1. Insert a row in the Locations table with *locationid* = 213 and *countryid* = NULL. Handle the exception regarding violating the integrity constraint (ORA-01400).

Challenge Problems

1. Create a PL/SQL block that raises an error if the user tries to execute it before 5 o'clock PM. Use both RAISE and RAISEAPPLICATIONERROR approaches.

Quick Reference Cheatsheet

Common Predefined Exceptions

Exception	Error Code	Description
NODATAFOUND	ORA-01403	SELECT INTO returns no rows
TOOMANYROWS	ORA-01422	SELECT INTO returns multiple rows
ZERO_DIVIDE	ORA-01476	Attempt to divide by zero
INVALID_NUMBER	ORA-01722	Conversion of string to number fails
DUPVALON_INDEX	ORA-00001	Unique constraint violation
INVALID_CURSOR	ORA-01001	Illegal cursor operation
CURSoralREADYOPEN	ORA-06511	Attempt to open already open cursor

Exception Handling Structure

```
EXCEPTION
  WHEN exception1 THEN
    -- Handler code
  WHEN exception2 THEN
    -- Handler code
```

```
WHEN OTHERS THEN
  -- Default handler
  code := SQLCODE;
  message := SQLERRM;
END;
```

Non-Predefined Exception Setup

```
DECLARE
  exception_name EXCEPTION;
  PRAGMA EXCEPTION_INIT(exception_name, error_code);
```

User-Defined Exception with RAISEAPPLICATIONERROR

```
DECLARE
  exception_name EXCEPTION;
  PRAGMA EXCEPTION_INIT(exception_name, -20xxx);
BEGIN
  IF condition THEN
    RAISE_APPLICATION_ERROR(-20xxx, 'Error message');
  END IF;
EXCEPTION
  WHEN exception_name THEN
    -- Handle error
END;
```

SQLCODE Values

- 0: No error
- 1: User-defined exception without code
- +100: NODATAFOUND exception
- Negative number: Oracle error code
- -20000 to -20999: Custom application errors

Best Practice Checklist

- ☐ Handle specific exceptions before WHEN OTHERS
- ☐ Log error details (SQLCODE, SQLERRM)
- ☐ Avoid empty exception handlers
- ☐ Consider user-friendly error messages
- ☐ Test exception handling paths
- ☐ Document expected exceptions

Seminar 7-8: Subprograms

Overview

This seminar covers PL/SQL subprograms, specifically procedures and functions, which are named PL/SQL blocks that can be stored in the database and reused. Subprograms are fundamental building blocks of modular database applications, promoting code reuse, maintainability, and encapsulation of business logic. Understanding how to create, manage, and use subprograms is essential for professional PL/SQL development.

Key Learning Objectives:

- Differentiate between procedures and functions

- Master parameter modes (IN, OUT, IN OUT)
- Learn to create, manage, and call subprograms
- Understand the scope and visibility rules for subprograms
- Apply best practices for effective subprogram design

Core Concepts

Subprogram Fundamentals

Subprograms are named PL/SQL blocks that can accept parameters and be called from other PL/SQL blocks or applications.

Key Characteristics:

- They can be grouped into program packages (PACKAGE)
- Global or substitution variables cannot be used in subprogram bodies
- Parameters are used to pass values to and from subprograms
- SHOW ERRORS is used to display compilation errors

Parameter Modes

Parameters define how data is passed between the calling program and the subprogram:

Mode	Description	Default	Usage
IN	Passes values to the subprogram (read-only)	Yes	Most common mode for input parameters
OUT	Returns values from the subprogram	No	Used for output parameters
IN OUT	Passes values to and returns values from the subprogram	No	Used when both input and output are needed

Subprogram Types

Type	Description	Return Value	Primary Use
Procedures	Perform actions or operations	None	Implement business logic, data manipulation
Functions	Compute and return a value	Required	Calculate values, transform data

Syntax Reference

Procedure Creation

```
CREATE [OR REPLACE] PROCEDURE procedure_name
    [(param1 [IN|OUT|IN OUT] type1,
      param2 [IN|OUT|IN OUT] type2, ...)]
IS | AS
    -- Local variable declarations
BEGIN
    -- Procedure Logic
    [EXCEPTION
      -- Exception handling]
END [procedure_name];
```

Function Creation

```
CREATE [OR REPLACE] FUNCTION function_name
    [(param1 [IN|OUT|IN OUT] type1,
      param2 [IN|OUT|IN OUT] type2, ...)]
RETURN return_type
IS | AS
    -- Local variable declarations
BEGIN
    -- Function Logic
    RETURN value;
    [EXCEPTION
        -- Exception handling]
END [function_name];
```

Subprogram Management

Action	Command
View procedures	SELECT object_name FROM user_objects WHERE object_type = 'PROCEDURE'
View procedure body	SELECT text FROM user_source WHERE name = 'PROCEDURE_NAME' AND type = 'PROCEDURE'
View functions	SELECT object_name FROM user_objects WHERE object_type = 'FUNCTION'
View function body	SELECT text FROM user_source WHERE name = 'FUNCTION_NAME' AND type = 'FUNCTION'
Delete procedure	DROP PROCEDURE procedure_name
Delete function	DROP FUNCTION function_name
View function return type	DESCRIBE function_name

Calling Subprograms

Method	Syntax	Notes
From PL/SQL block	procedure_name(parameters);	Most common method
Direct call (SQL*Plus)	EXEC [UTE] procedure_name(parameters);	Creates anonymous block
CALL statement	CALL procedure_name(parameters);	SQL command, different behavior
From other environments	Varies by environment	APEX, Oracle Developer Suite, etc.

Examples

Basic Example: Procedure with IN Parameters

```
CREATE OR REPLACE PROCEDURE modify_salary(
    v_employee_id IN employees.employee_id%TYPE,
    percent NUMBER
)
IS
    v_salary employees.salary%TYPE;
BEGIN
    SELECT salary INTO v_salary
```

```

FROM employees
WHERE employee_id = v_employee_id;

DBMS_OUTPUT.PUT_LINE('The employee has the salary: ' || v_salary);

UPDATE employees
SET salary = salary * (1 + percent/100)
WHERE employee_id = v_employee_id;

SELECT salary INTO v_salary
FROM employees
WHERE employee_id = v_employee_id;

DBMS_OUTPUT.PUT_LINE('The employee has the salary: ' || v_salary);
END;
/

-- Possible calls:
CALL modify_salary(176, 10);
-- OR
EXECUTE modify_salary(176, 10);
-- OR
BEGIN
    modify_salary(176, 10);
END;
/

```

Basic Example: Procedure with OUT Parameters

```

CREATE OR REPLACE PROCEDURE proc_employee(
    p_employee_id IN employees.employee_id%TYPE,
    name OUT employees.first_name%TYPE,
    p_salary OUT employees.salary%TYPE
)
IS
BEGIN
    SELECT first_name, salary
    INTO name, p_salary
    FROM employees
    WHERE employee_id = p_employee_id;
END;
/

-- Call:
DECLARE
    name employees.first_name%TYPE;
    v_salary employees.salary%TYPE;
BEGIN
    proc_employee(150, name, v_salary);
    DBMS_OUTPUT.PUT_LINE('The employee ' || name || ' has the salary: ' || v_salary);
END;
/

```

Basic Example: Procedure with IN OUT Parameters

```

CREATE OR REPLACE PROCEDURE modify_avg_salary(
    avg_salary IN OUT NUMBER
)
IS
    CURSOR c IS
        SELECT employee_id, salary
        FROM employees
        WHERE salary < avg_salary;
BEGIN
    FOR var IN c LOOP

```

```

        modify_salary(var.employee_id, 15);
    END LOOP;

    SELECT AVG(salary) INTO avg_salary
    FROM employees;
END;
/

-- Call:
DECLARE
    v_avg_salary NUMBER;
BEGIN
    SELECT AVG(salary) INTO v_avg_salary
    FROM employees;

    DBMS_OUTPUT.PUT_LINE('The initial average salary: ' || ROUND(v_avg_salary));

    modify_avg_salary(v_avg_salary);

    DBMS_OUTPUT.PUT_LINE('The final average salary: ' || ROUND(v_avg_salary));
END;
/

```

Basic Example: Function

```

CREATE OR REPLACE FUNCTION check_salary(
    p_employee_id IN employees.employee_id%TYPE
)
RETURN BOOLEAN
IS
    v_salary employees.salary%TYPE;
    average_salary NUMBER;
BEGIN
    SELECT AVG(salary) INTO average_salary
    FROM employees;

    DBMS_OUTPUT.PUT_LINE('avg sal: ' || average_salary);

    SELECT salary INTO v_salary
    FROM employees
    WHERE employee_id = p_employee_id;

    DBMS_OUTPUT.PUT_LINE('salary: ' || v_salary);

    IF v_salary > average_salary THEN
        RETURN TRUE;
    ELSE
        RETURN FALSE;
    END IF;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN NULL;
END;
/

-- Call:
BEGIN
    IF check_salary(11) THEN
        DBMS_OUTPUT.PUT_LINE('the salary is higher');
    ELSIF (NOT check_salary(11)) THEN
        DBMS_OUTPUT.PUT_LINE('the salary is smaller');
    ELSE
        DBMS_OUTPUT.PUT_LINE('the employee doesn't exist');
    END IF;
END;
/

```

Common Patterns & Best Practices

Subprogram Design Guidelines

- **Procedures** should perform actions without returning values
- **Functions** should compute and return values without side effects
- Keep subprograms focused on a single responsibility
- Use meaningful names that reflect the subprogram's purpose
- Document parameters and expected behavior

Parameter Usage Best Practices

- Use **IN** parameters for input values that shouldn't be modified
- Use **OUT** parameters when returning multiple values
- Use **IN OUT** parameters sparingly (often indicates design issues)
- Avoid excessive numbers of parameters (consider records for grouping)
- Document parameter expectations and constraints

Error Handling in Subprograms

- Handle exceptions within subprograms when appropriate
- Consider whether to handle or propagate exceptions
- Use `RAISEAPPLICATIONERROR` for meaningful error messages
- Document expected exceptions in comments
- Consider creating a standardized error handling approach

Performance Considerations

- Minimize context switching between SQL and PL/SQL
- Use bulk operations when processing multiple rows
- Avoid unnecessary computations in frequently called subprograms
- Consider deterministic functions for optimization opportunities
- Be aware of the performance implications of remote procedure calls

Common Pitfalls to Avoid

- Using global variables instead of parameters
- Creating overly complex subprograms with too many responsibilities
- Ignoring error handling in subprograms
- Using functions with side effects (violates expected behavior)
- Creating subprograms with excessive parameters

Exercises

Foundational Exercises

1. Create a procedure that modifies the salary of an employee whose ID is given as a parameter. Also, the percentage of salary growth is given as a parameter.

Applied Exercises

1. Create a procedure that returns the name and the salary of an employee whose ID is given.

2. Create a procedure that calculates the average salary and returns it using a host variable.

Challenge Problems

1. Create a procedure that modifies the salary of all employees if their salary is smaller than the average salary. The procedure receives the average salary and returns it after the updates have been made. It calls the modify_salary procedure and uses 15 for the percentage.

Quick Reference Cheatsheet

Parameter Modes

Mode	Direction	Default Value	Can Be Modified	Usage Notes
IN	In only	Yes	No	Most common for input
OUT	Out only	No	Yes	Must be assigned a value
IN OUT	In and out	No	Yes	Rarely needed

Subprogram Creation

```
-- Procedure
CREATE OR REPLACE PROCEDURE name (params) IS
BEGIN
  -- Logic
END;

-- Function
CREATE OR REPLACE FUNCTION name (params) RETURN type IS
BEGIN
  -- Logic
  RETURN value;
END;
```

Calling Methods

```
-- From PL/SQL
name(params);

-- From SQL*Plus
EXEC name(params);

-- SQL CALL statement
CALL name(params);
```

Management Commands

```
-- View subprograms
SELECT object_name FROM user_objects
WHERE object_type IN ('PROCEDURE', 'FUNCTION');

-- View source
SELECT text FROM user_source
WHERE name = 'NAME' AND type IN ('PROCEDURE', 'FUNCTION')
ORDER BY line;

-- Drop subprogram
DROP PROCEDURE name;
DROP FUNCTION name;
```

Best Practices Checklist

- ☐ Use appropriate parameter modes

- ☐ Keep subprograms focused and single-purpose
 - ☐ Document parameters and expected behavior
 - ☐ Handle errors appropriately
 - ☐ Avoid global variables
 - ☐ Test with boundary conditions
 - ☐ Consider performance implications
-

Seminar 9: Triggers

Overview

This seminar covers database triggers in PL/SQL, which are special stored procedures that automatically execute in response to specified events. Triggers provide a powerful mechanism for implementing business rules, maintaining data integrity, and automating database operations. Understanding trigger types, execution timing, and best practices is essential for developing robust database applications.

Key Learning Objectives:

- Understand the concept and purpose of database triggers
- Master the different types of triggers (BEFORE, AFTER, INSTEAD OF)
- Learn to create and manage row-level and statement-level triggers
- Work with :OLD and :NEW pseudo-records in row-level triggers
- Apply best practices for effective trigger design and implementation

Core Concepts

Trigger Fundamentals

A trigger is a PL/SQL subprogram that automatically executes when a specified event occurs.

Key Characteristics:

- Associated with a table, view, schema, or database
- Stored in the database
- Executed automatically when triggering events occur
- Used for managing complex integrity constraints, monitoring, and centralizing operations

Trigger Components

When declaring a trigger, the following components must be specified:

Component	Description
Trigger Type	BEFORE, AFTER, or INSTEAD OF (for views)
Triggering Event	INSERT, UPDATE, DELETE, or schema/database events
Trigger Level	ROW (executed for each affected row) or STATEMENT (executed once per statement)
Triggering Condition	Optional condition that determines if the trigger should fire

Trigger Action	PL/SQL block or procedure call that executes when the trigger fires

Trigger Types by Event

Event Category	Trigger Types	Description
DML Triggers	BEFORE/AFTER INSERT/UPDATE/DELETE	Execute in response to table modifications
INSTEAD OF Triggers	INSTEAD OF	Execute for views that aren't otherwise updatable
DDL Triggers	BEFORE/AFTER CREATE/ALTER/DROP	Execute in response to schema changes
Database Event Triggers	BEFORE/AFTER LOGON/LOGOFF/SHUTDOWN	Execute in response to database events

Pseudo-Records in Row-Level Triggers

Row-level triggers can access the current row using two pseudo-records:

Operation	:OLD Value	:NEW Value
INSERT	NULL	New value being inserted
UPDATE	Old value before update	New value after update
DELETE	Old value before delete	NULL

Syntax Reference

Trigger Creation Syntax

```
CREATE [OR REPLACE] TRIGGER trigger_name
  [BEFORE | AFTER | INSTEAD OF]
  [INSERT | UPDATE [OF column_list] | DELETE]
  ON table_name
  [FOR EACH ROW]
  [WHEN (condition)]
DECLARE
  -- Variable declarations
BEGIN
  -- Trigger logic
  [EXCEPTION
    -- Exception handling]
END;
```

Trigger Management Commands

Action	Command
View triggers	SELECT trigger_type, trigger_name, triggering_event FROM user_triggers WHERE table_name = 'TABLE_NAME'
Disable trigger	ALTER TRIGGER trigger_name DISABLE
Enable trigger	ALTER TRIGGER trigger_name ENABLE
Disable all table triggers	ALTER TABLE table_name DISABLE ALL TRIGGERS

Enable all table triggers	ALTER TABLE table_name ENABLE ALL TRIGGERS
Recompile trigger	ALTER TRIGGER trigger_name COMPILE
Delete trigger	DROP TRIGGER trigger_name

Examples

Basic Example: Simple Trigger

```
CREATE OR REPLACE TRIGGER products_trig
BEFORE INSERT
ON product_information
BEGIN
    DBMS_OUTPUT.PUT_LINE('Trigger executed');
END;
/
```

Basic Example: Row-Level Trigger with WHEN Clause

```
CREATE OR REPLACE TRIGGER restrict_price
BEFORE INSERT OR UPDATE ON product_information
FOR EACH ROW
WHEN (NEW.product_status = 'orderable')
BEGIN
    IF :NEW.list_price < :NEW.min_price THEN
        RAISE_APPLICATION_ERROR(-20202, 'The price has to be higher than the min price!');
    END IF;
END;
/

UPDATE product_information
SET list_price = min_price - 1
WHERE product_id = 2384;
```

Basic Example: Trigger with Operation Detection

```
CREATE OR REPLACE TRIGGER products_log
BEFORE INSERT OR UPDATE OR DELETE ON product_information
DECLARE
    v_type VARCHAR2(1);
BEGIN
    CASE
        WHEN INSERTING THEN v_type := 'I';
        WHEN UPDATING THEN v_type := 'U';
        ELSE v_type := 'D';
    END CASE;

    DBMS_OUTPUT.PUT_LINE(USER || ' ' || SYSDATE || ' ' || v_type);
END;
/
```

Advanced Example: INSTEAD OF Trigger for View

```
-- For a view built from multiple tables
CREATE OR REPLACE TRIGGER view_update_trigger
INSTEAD OF UPDATE ON view_name
FOR EACH ROW
BEGIN
    -- Update underlying tables
    UPDATE table1 SET column1 = :NEW.column1 WHERE id = :OLD.id;
    UPDATE table2 SET column2 = :NEW.column2 WHERE id = :OLD.id;
```

```
END;  
/
```

Advanced Example: Audit Trigger

```
-- Create audit table  
CREATE TABLE temp_log (  
    operation_type VARCHAR2(10),  
    user_name VARCHAR2(30),  
    change_date DATE,  
    product_id NUMBER  
);  
  
-- Create trigger  
CREATE OR REPLACE TRIGGER products_audit  
AFTER INSERT OR DELETE OR UPDATE ON products  
FOR EACH ROW  
BEGIN  
    IF INSERTING THEN  
        INSERT INTO temp_log VALUES ('INSERT', USER, SYSDATE, :NEW.product_id);  
    ELSIF DELETING THEN  
        INSERT INTO temp_log VALUES ('DELETE', USER, SYSDATE, :OLD.product_id);  
    ELSIF UPDATING THEN  
        INSERT INTO temp_log VALUES ('UPDATE', USER, SYSDATE, :OLD.product_id);  
    END IF;  
END;  
/
```

Common Patterns & Best Practices

Trigger Design Guidelines

- Use triggers for business rules that must always be enforced
- Keep trigger logic simple and focused
- Avoid complex processing in triggers
- Document trigger purpose and behavior thoroughly
- Consider alternatives (constraints, application logic) before using triggers

When to Use Triggers

- Enforcing complex business rules that can't be handled with constraints
- Maintaining derived or historical data
- Implementing security restrictions
- Creating audit trails
- Synchronizing data across tables

When to Avoid Triggers

- When the same functionality can be achieved with constraints
- For complex business logic that might be better handled in application code
- When performance is critical (triggers add overhead)
- When the logic might change frequently
- When the same rule needs different behavior in different contexts

Performance Considerations

- Row-level triggers execute once per row (can be expensive for bulk operations)
- Statement-level triggers execute once per statement (more efficient)

- Avoid SQL statements within row-level triggers
- Consider using compound triggers for bulk operations
- Be aware of the potential for "mutating table" errors

Common Pitfalls to Avoid

- Creating recursive triggers (a trigger that causes itself to fire again)
- Ignoring the order of trigger execution
- Assuming trigger execution order for multiple triggers on the same event
- Using triggers for tasks better handled by constraints
- Creating overly complex triggers that are difficult to maintain

Exercises

Foundational Exercises

1. Create a trigger that does not allow the update of the credit limit of customers between the 28th and 31st of the month (in view of the closing of the monthly financial statements).

Applied Exercises

1. Create a trigger that does not allow the existence of negative salary values.
2. Add a Stock column to the Products table. Enter the value 2000 in the newly added column. Create a trigger that does not allow ordering a product in a quantity greater than the related stock. At the same time, for the ordered products, the trigger will decrease the stock by the ordered value.

Challenge Problems

1. Create a table TEMP_LOG that will be populated during INSERT, DELETE, or UPDATE operations on the Products table, with the type of operation, the user who executed it, and the current date.

Quick Reference Cheatsheet

Trigger Creation Template

```
CREATE OR REPLACE TRIGGER trigger_name
[BEFORE | AFTER | INSTEAD OF]
[INSERT | UPDATE [OF column] | DELETE]
ON table_name
[FOR EACH ROW]
[WHEN (condition)]
BEGIN
  -- Trigger logic
END;
```

Pseudo-Records Reference

Operation	:OLD	:NEW
INSERT	NULL	New values
UPDATE	Old values	New values
DELETE	Old values	NULL

Trigger Management

```
-- View triggers
SELECT trigger_name, triggering_event, status
```

```
FROM user_triggers
WHERE table_name = 'TABLE';

-- Disable/Enable
ALTER TRIGGER trigger_name DISABLE|ENABLE;
ALTER TABLE table_name DISABLE|ENABLE ALL TRIGGERS;

-- Recompile
ALTER TRIGGER trigger_name COMPILE;

-- Delete
DROP TRIGGER trigger_name;
```

Special Trigger Conditions

```
-- Detect operation type
IF INSERTING THEN ... END IF;
IF UPDATING THEN ... END IF;
IF DELETING THEN ... END IF;

-- Column-specific update
WHEN (UPDATING('column_name'))
```

Best Practice Checklist

- ☐ Keep trigger logic simple
- ☐ Document trigger purpose clearly
- ☐ Consider performance implications
- ☐ Test trigger with all relevant operations
- ☐ Avoid recursive triggers
- ☐ Use appropriate trigger timing (BEFORE/AFTER)
- ☐ Prefer statement-level triggers when possible
- ☐ Handle exceptions appropriately

This restructuring maintains all technical content while significantly improving the organization, clarity, and pedagogical value of the material. Each seminar now follows a logical progression from concepts to practical application, with clear examples and exercises that build understanding progressively.